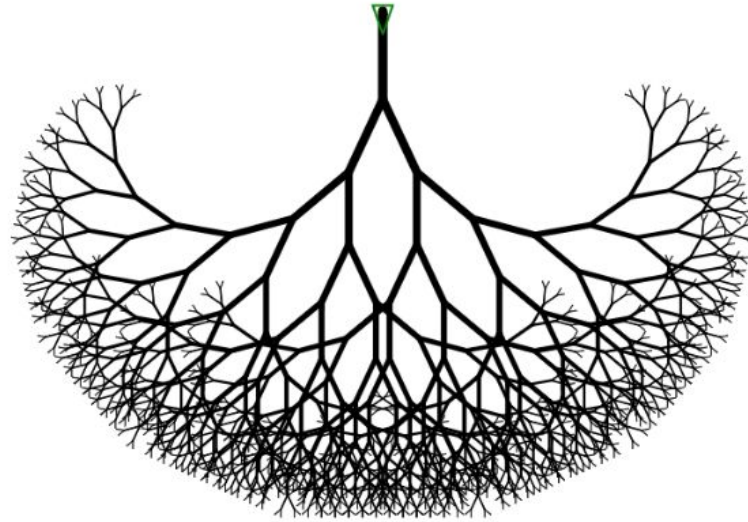


Trees



Lecture Flow

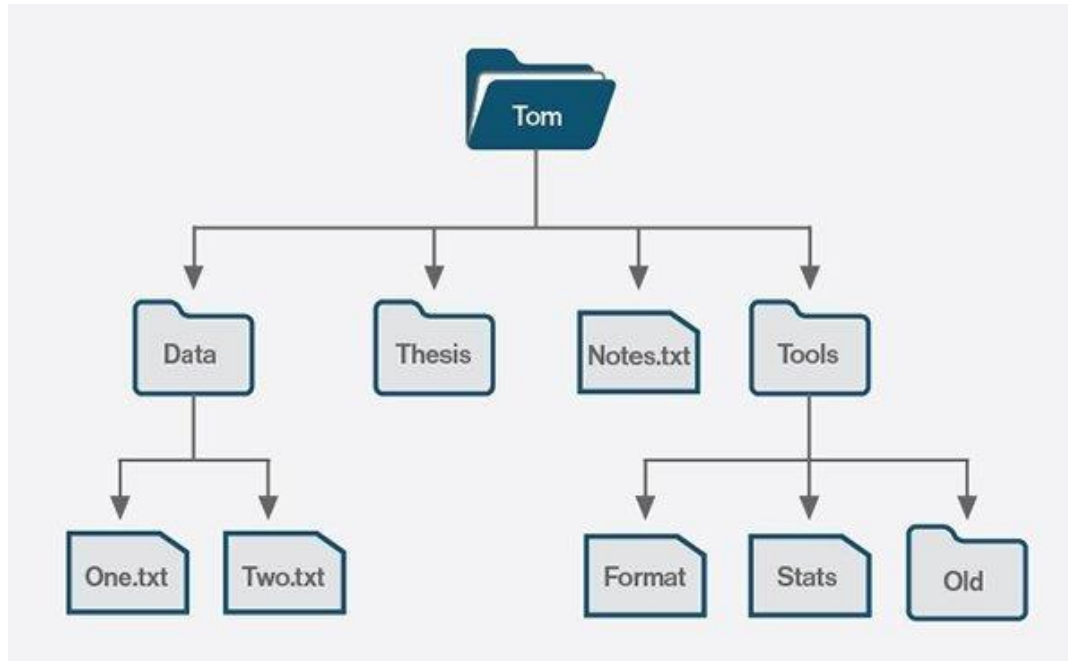
1. Prerequisites
2. Real life problem
3. Definition
4. Tree Terminologies
5. Types of Trees
6. Tree Traversal
7. Basic Tree Operations
8. Checkpoint
9. Time and space complexity analysis
10. Things to pay attention to (common pitfalls)
11. Applications of a Tree
12. Practice Questions

Pre-requisites

- Basic Recursion
- Linked List
- Stacks
- Queues

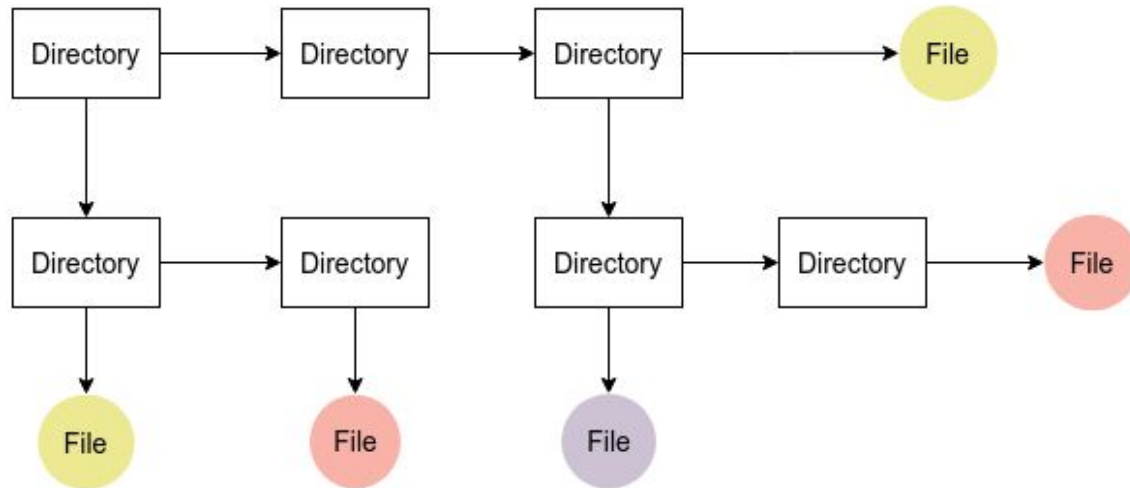
Real life problem

What data structure will be best to implement a file management system ?

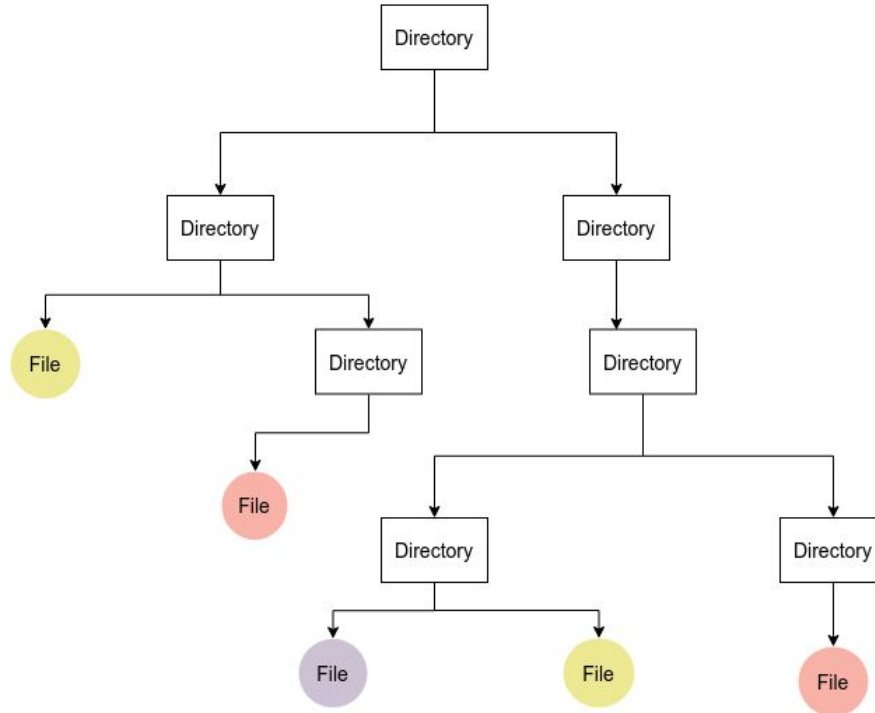


- Considering all these cases, we know that an array, stack, or queue will not work. Can we implement this with the help of a linked list? We could make a linked list in such a way that if there is a directory at any particular node, we create a new linked list from here.

What will this look like?



- It looks similar to a tree, right? So, we tried to use a linked list but ended up using a tree because the problem can easily be solved using **tree** data structure.
- Each internal node of the tree will be a directory, and each leaf node will be a file.

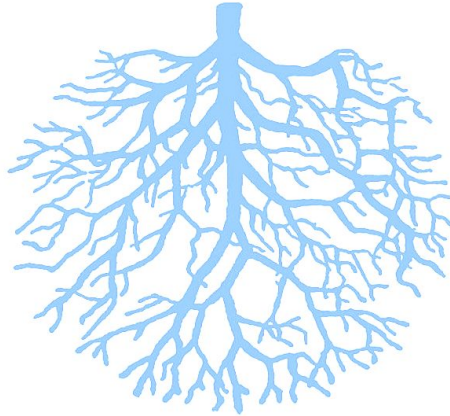


Definition

- Unlike linear data structures (linked list, stack, queue), a tree is a hierarchical, non-linear data structure composed of one or more nodes (or no node at all).
- Trees are all around us. What does a typical tree look like?
 - Leaves
 - Branches

...definition

- Trees in data structures have their roots on the top and the leaves below. They are upside-down trees.



Formal Definition of A tree

A tree is a **connected, undirected graph with no cycles**. In a tree, there are no cycles, which means that there is only **one possible path** between any two nodes.

Tree Terminologies

Node

- A node is a structure that may contain a value or a condition or a separate data structure like an array, linked list, or even a tree. Each node of the tree can have zero or more children, but it can have only one parent, and if the node is a root node, it will not have any parent.

```
class Node:

    def __init__(self, key):

        self.left = None

        self.right = None

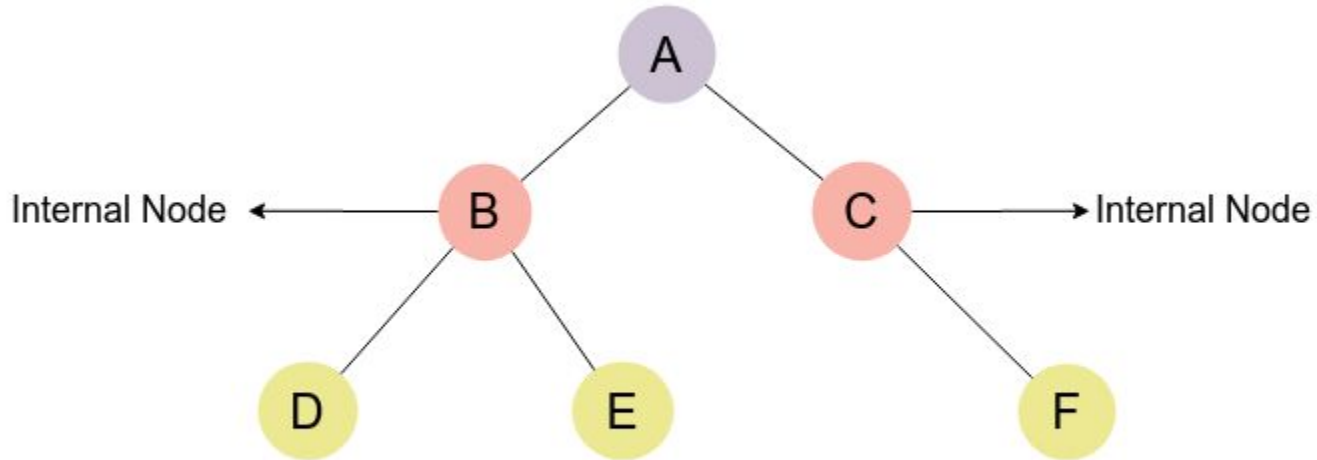
        self.val = key
```

Root Node(Root)

- A root is the first node of the tree, or we can say the node that does not have any parent. If the root node is connected to any other node, then that node is called the immediate child of the root node.

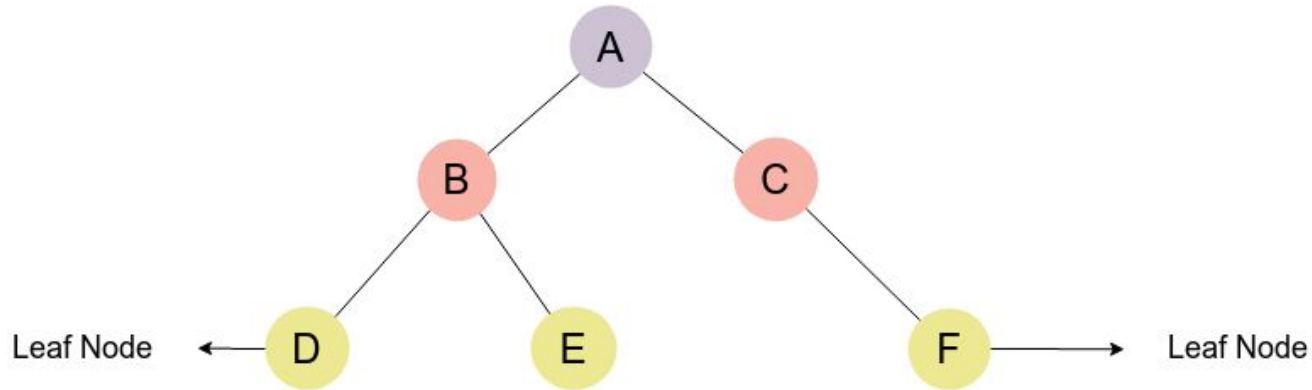
Internal Node or Inner Nodes

- Each node with a parent and a child is called the inner node or internal node of the tree.



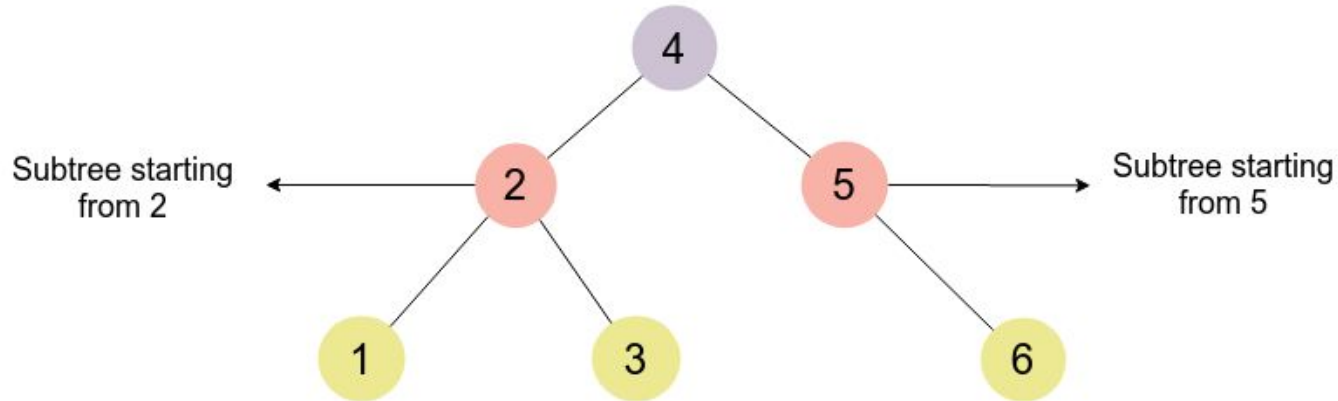
Leaf Node

- Nodes that do not have any further child nodes are called leaf nodes of the tree. A root node that does not have any children can also be called the only leaf node of the tree.

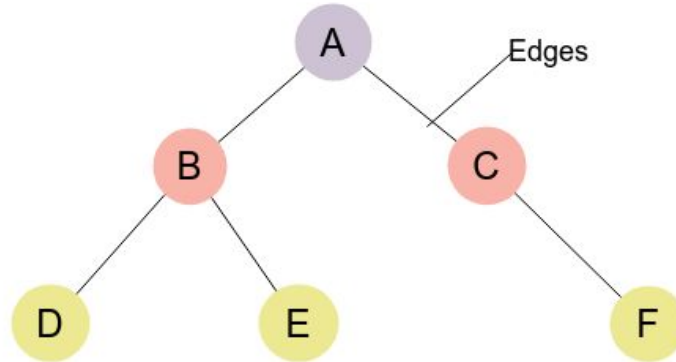


Subtree of a Tree

- Each node of a tree forms a recursive subtree of the tree. A subtree of a tree consists of a node **n** and all of the descendants of node **n**.

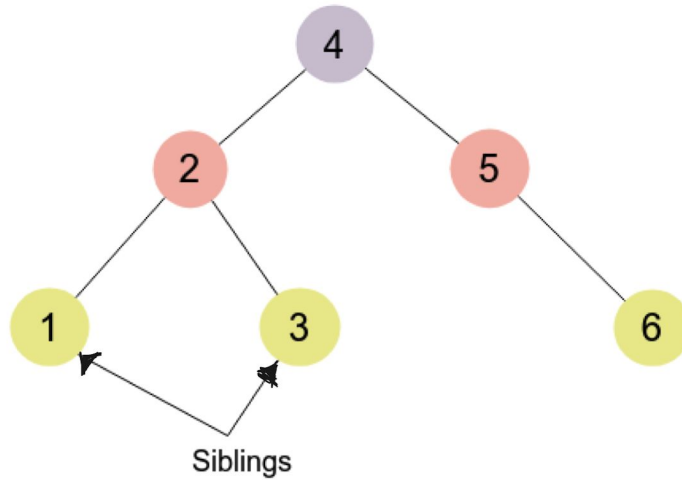


- **Parent Node:** The node which is a predecessor of a node is called the parent node of that node.
- **Child Node:** The node which is the immediate successor of a node is called the child node of that node.
- **Edge:** Edge acts as a link between the parent node and the child node.



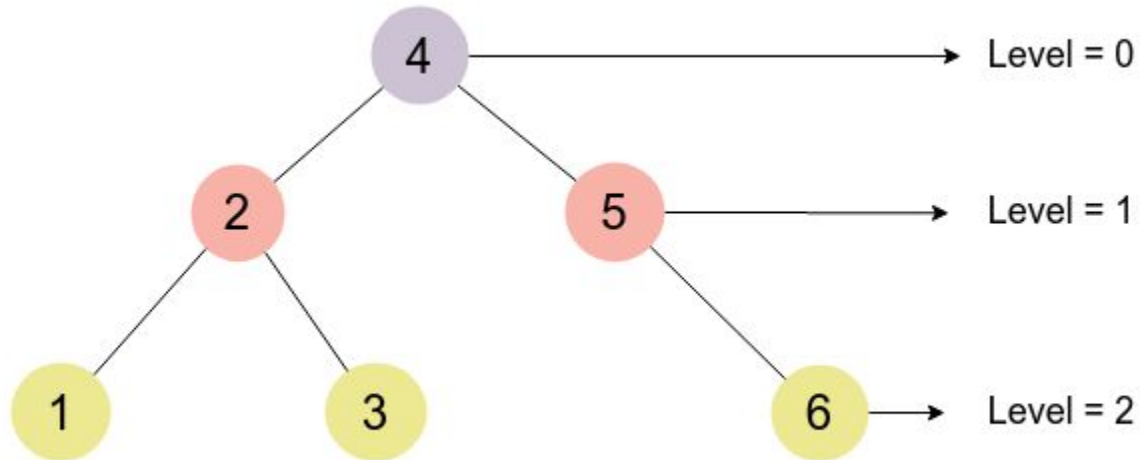
Siblings

- In a tree, two or more nodes that share the same parent node are called sibling nodes.



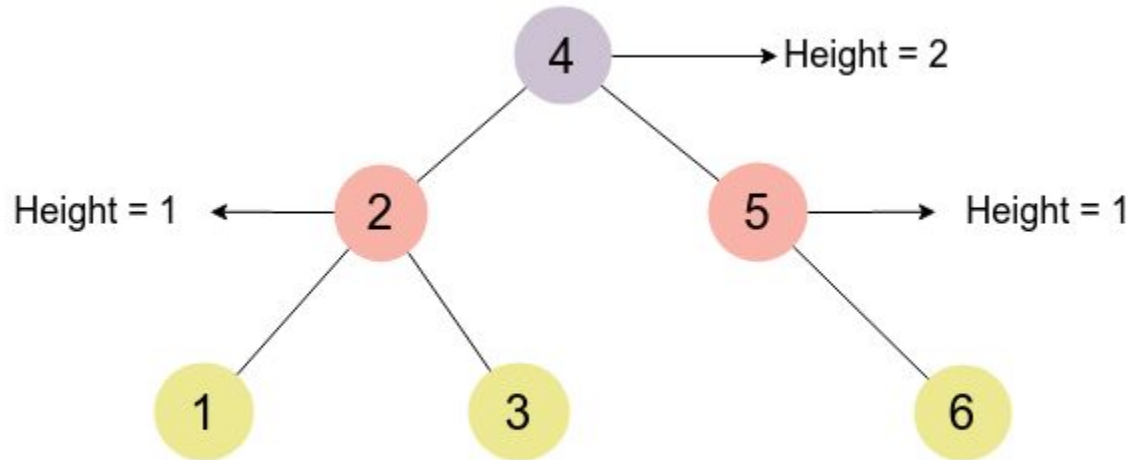
Level of a Tree

- Level of the tree indicates how far you have gone from the root node. The level of the tree starts with 0 and increments by one as you are going from top to bottom.



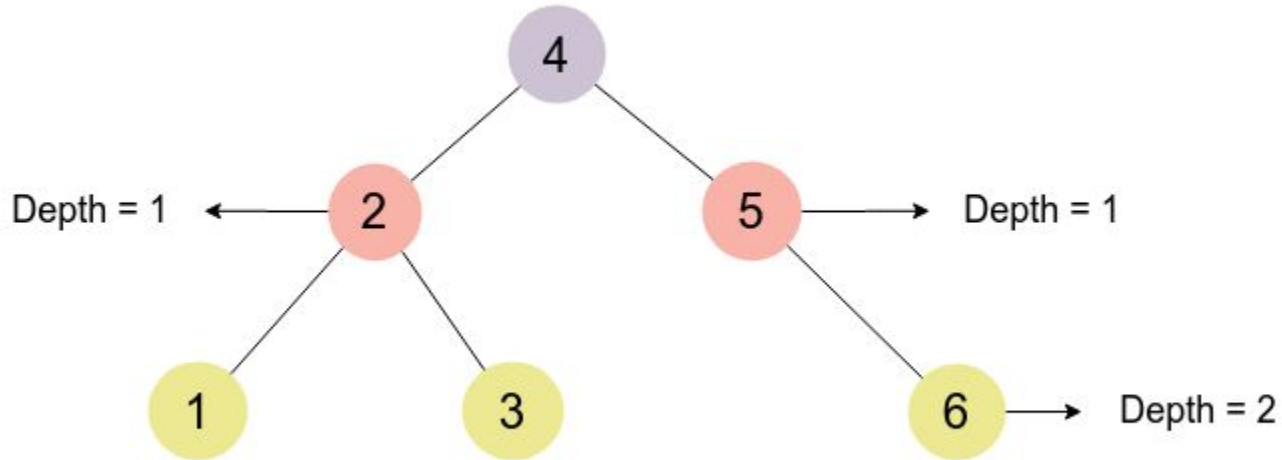
Height of a Tree

- The tree's height is the total number of edges from the root node to the farthest leaf node of the tree.



Depth of a Tree

- The depth of the tree is similar to the height; the only difference is that it goes backward. It means the depth of the node is the total number of edges from the root node to the current node. It is very similar to the level of the node.



Types of Trees

Types of Trees

- In data structures, there are various types of trees. The selection of trees depends upon the nature of the problems we are trying to solve. These are the basic types of trees:
 - Binary Tree
 - Complete Binary Tree
 - Balanced Binary Tree
 - Binary Search Tree
 - Ternary Tree
 - N-ary Tree

Node Implementation of Binary and Nary Tree

Binary Tree

```
class Node:

    def __init__(self, key):

        self.left = None

        self.right = None

        self.val = key
```

N-ary Tree

```
class Node:

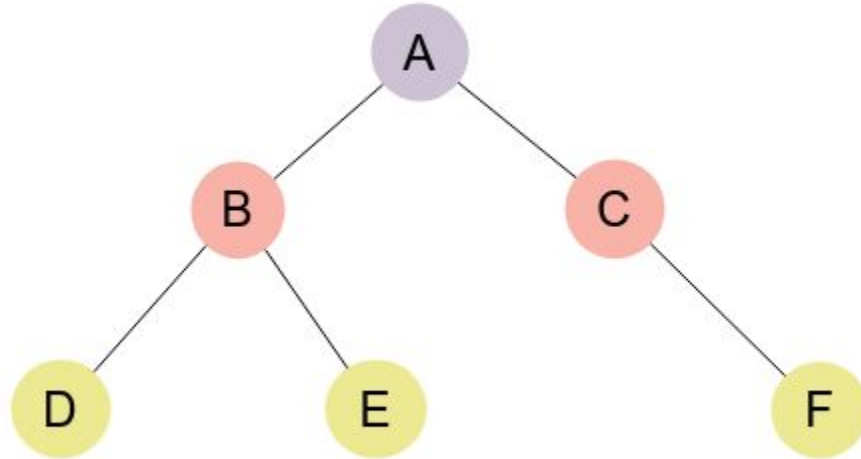
    def __init__(self, val=None, children=[]):

        self.val = val

        self.children = children
```

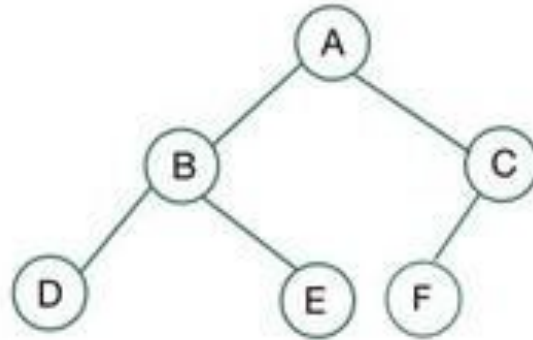
Binary Tree

- A binary tree is a tree in which every internal node and root node has at most two children. These two child nodes are often called the left child node or right child node of the node.



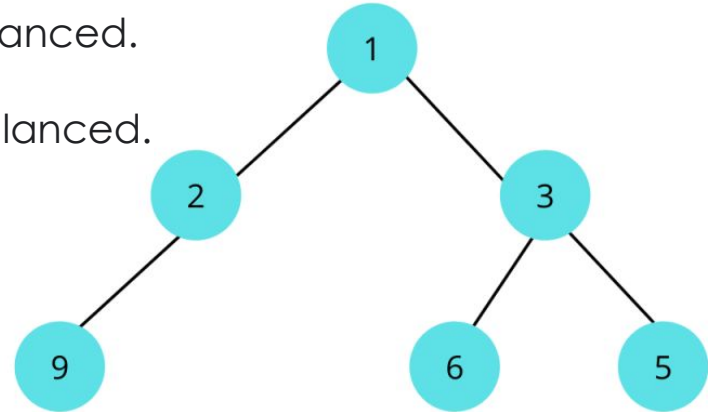
Complete Binary Tree

- A complete binary tree is a special type of binary tree where all the levels of the tree are filled completely except the lowest level nodes which are filled from as left as possible.



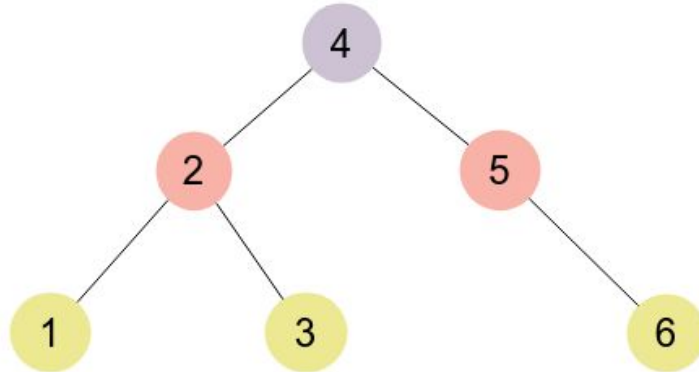
Balanced Binary Tree

- A balanced binary tree is a binary tree that follows the 3 conditions:
 - The height of the left and right tree for any node does not differ by more than 1.
 - The left subtree of that node is also balanced.
 - The right subtree of that node is also balanced.



Binary Search Tree(BST)

- A binary search tree is a binary tree that has the following properties:
 - The left subtree of the node only contains values less than the value of the node.
 - The right subtree of the node only contains values greater than or equal to the value of the node.
 - The left and right subtree of the nodes should also be the binary search tree.



BST Continued

- The properties that separate a binary search tree from a regular binary tree is:
 1. All nodes of left subtree are less than the root node
 2. All nodes of right subtree are more than the root node
 3. Both subtrees of each node are also BSTs i.e. they have the above two properties

Tree Traversal

[Binary Tree Traversal](#)

Tree Traversal

- Traversing a tree means visiting every node in the tree. You might, for instance, want to add all the values in the tree or find the largest one. For all these operations, you will need to visit each node of the tree.
- Linear data structures like arrays, stacks, queues, and linked lists have only one way to read the data. But a hierarchical data structure like a tree can be traversed in different ways.

Tree Traversal

- If you implement a tree, you need a way to perform certain operations like finding the node's depth, height of the node, order each node of the tree (from left to right), etc.
- All these operations can be performed easily with the help of tree traversal algorithms.

DFS (Depth First Search)

- Depth-first search (DFS) is a method for exploring a tree or graph. In a DFS, you go as deep as possible down one path before backing up and trying a different one. You explore one path, hit a dead end, and go back and try a different one.
- There are basically three ways of traversing a binary tree:
 1. **Inorder Traversal**
 2. **Preorder Traversal**
 3. **Postorder Traversal**
- These traversal techniques are not only limited to the binary tree; they can easily be modified for an n-ary tree (a tree with a maximum of n children).

Preorder Traversal (Root, Left, Right)

- In preorder traversal, we traverse the root node first, then the left subtree of the binary tree, and finally, the node's right subtree.
- Preorder traversal is used to create a copy of the tree. Preorder traversal is also used to get prefix expressions on an expression tree.

```
# A function to do preorder tree traversal
def printPreOrder(root):
    if root:
        # First print the data of node
        print(root.val),
        # Then recur on left child
        printPreOrder(root.left)
        # Finally recur on right child
        printPreOrder(root.right)
```

Inorder Traversal (Left, Root, Right)

- Visit the current node before visiting any nodes inside the left or right subtrees. Here, the traversal is root – left child – right child. It means that the root node is traversed first then its left child and finally the right child.
- In the case of binary search trees (BST), Inorder traversal gives nodes in non-decreasing order. To get nodes of BST in non-increasing order, a variation of Inorder traversal where Inorder traversal is reversed can be used.

```
# A function to do inorder tree traversal

def printInorder (root):

    if root:

        # First recur on left child

        printInorder (root.left)

        # then print the data of node

        print(root.val),

        # now recur on right child

        printInorder (root.right)
```

Post Order Traversal (Root, Left, Right)

- In postorder traversal, we traverse the left subtree of the node and then the right subtree tree of the node, and finally, the root node of the tree.

```
# A function to do postorder tree traversal

def printPostOrder(root):

    if root:

        # First recur on left child
        printPostOrder(root.left)

        # the recur on right child
        printPostOrder(root.right)

        # now print the data of node
        print(root.val)
```

Practice

[Postorder-traversal](#)

[Preorder-traversal](#)

[Inorder-traversal](#)

[Maximum Depth of Binary Tree](#)

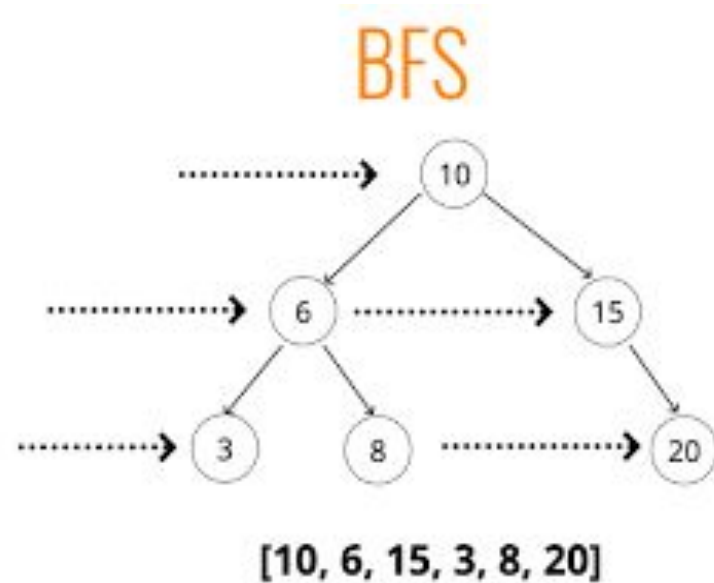
Breadth First Search Algorithm (Level order traversal)

It starts at the root node and visits all the nodes at the same level before moving on to the next level.

Algorithm for BFS in a Tree:

1. Create a queue and initialize it with the root node.
2. Loop until the queue is empty:
 - a. Remove the node at the front of the queue and visit it.
 - b. Add all its children to the queue.
3. Finish.

-



BFS Algorithm

```
def bfs(root: Node):  
  
    Queue = deque([Node])  
  
    While queue:  
  
        currNode = queue.popleft()  
  
        if currNode:  
  
            print(currNode.val)  
  
            queue.append(currNode.left)  
  
            queue.append(currNode.right)
```

```
def bfs(root: Node):  
  
    Queue = deque([Node])  
  
    While queue:  
  
        currNode = queue.popleft()  
  
        print(currNode.val)  
  
        for child in currNode.children:  
  
            queue.append(child)
```

Process all at the same level at once

```
def bfs(root: Node):  
    Queue = [Node]  
    While queue:  
        nextLevel = []  
        for currNode in queue:  
            print(currNode.val)  
            for child in currNode.children:  
                nextLevel.append(child)  
        queue = nextLevel
```

Advantages of BFS

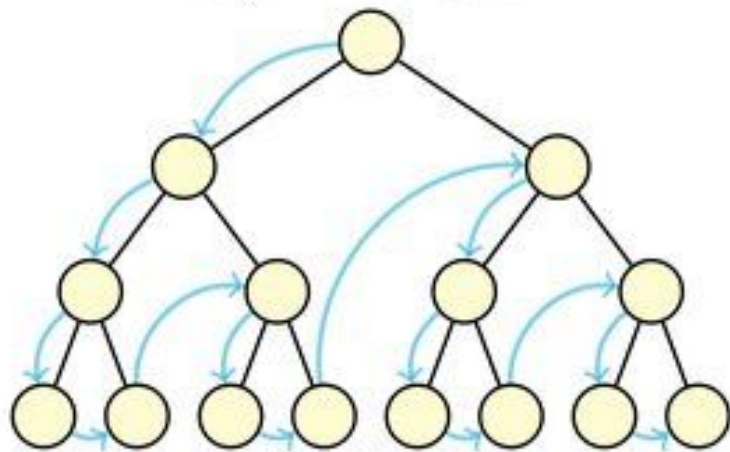
- BFS visits all the nodes **at a given depth** before moving on to the nodes at the next depth. Therefore, it is useful for searching a tree or graph for **the shortest path** or the **minimum number of steps required to reach a particular node**.
- BFS can also be used to find all the nodes at a given depth or level

BFS

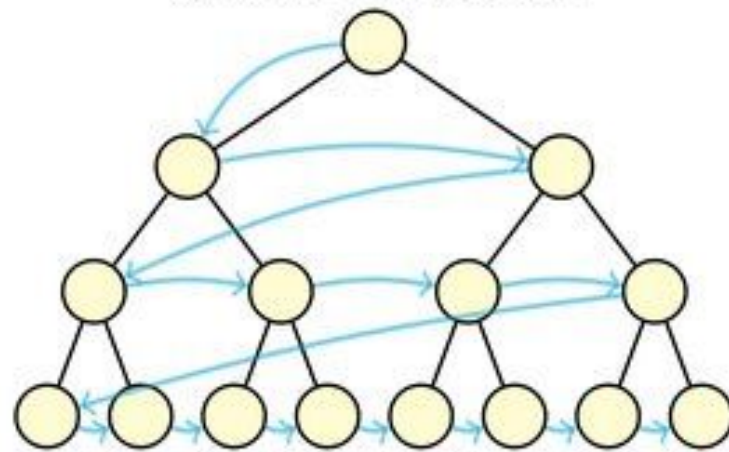
- Minimum depth of binary tree

Quick reminder

Depth-First Search



Breadth-First Search



Basic Operations on Trees(BST)

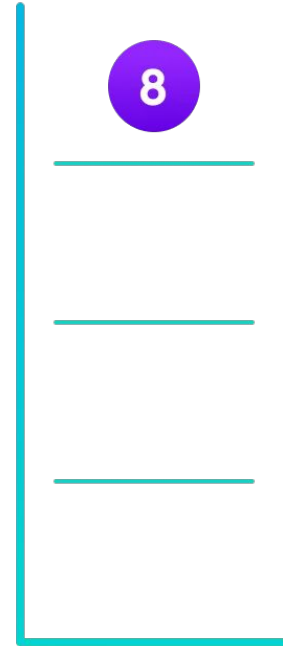
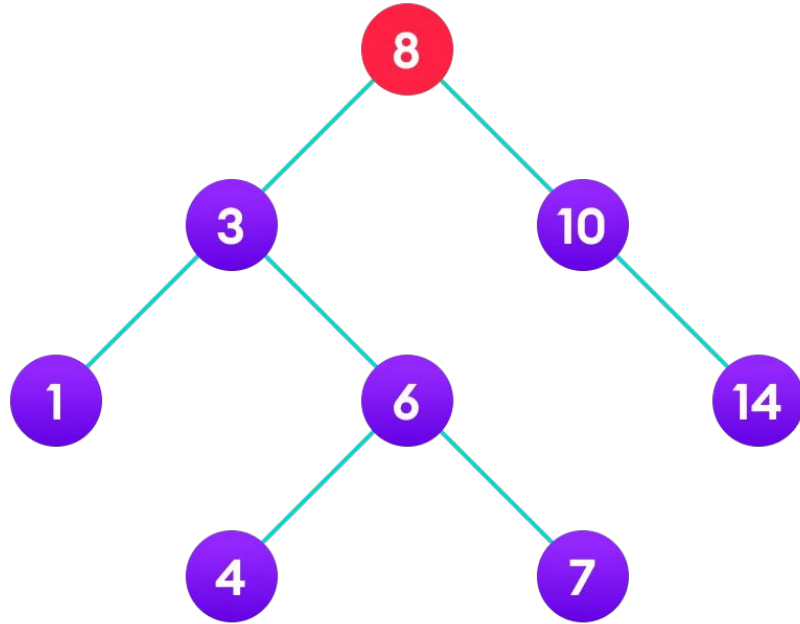
[Search in a Binary Search Tree](#)

Search Operation

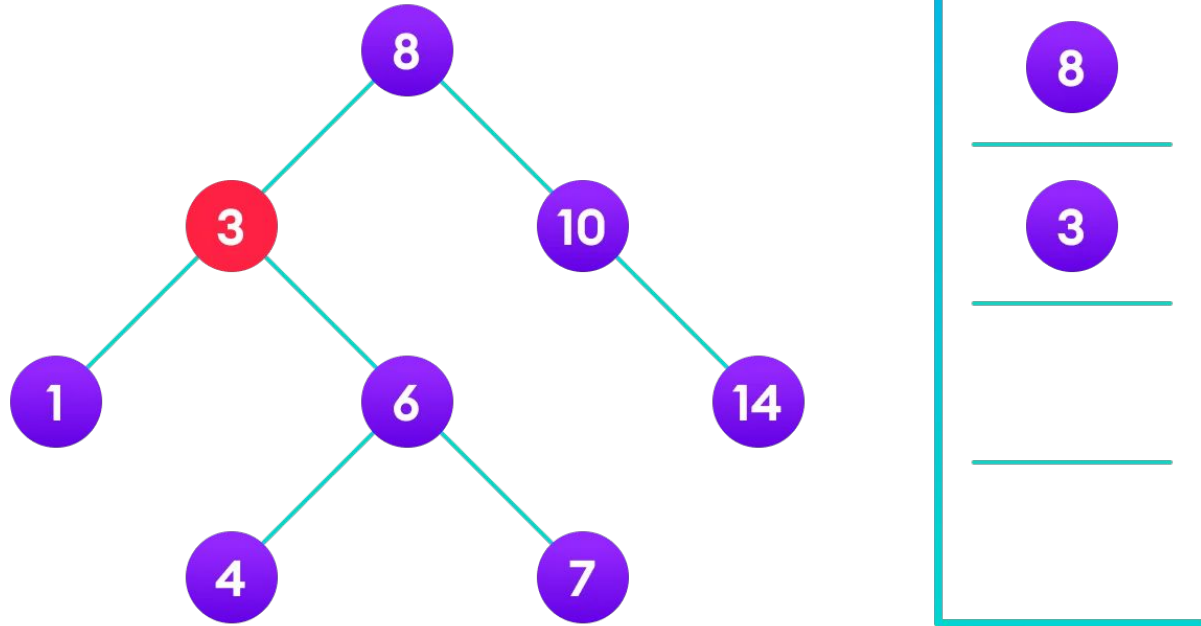
- The algorithm depends on the property of BST that if each left subtree has values below root and each right subtree has values above the root.
- If the value is below the root, we can say for sure that the value is not in the right subtree; we need to only search in the left subtree and if the value is above the root, we can say for sure that the value is not in the left subtree; we need to only search in the right subtree.

Let us try to visualize this with a diagram searching for 4 in the tree:

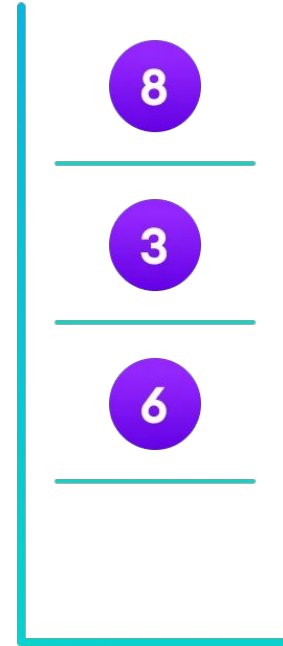
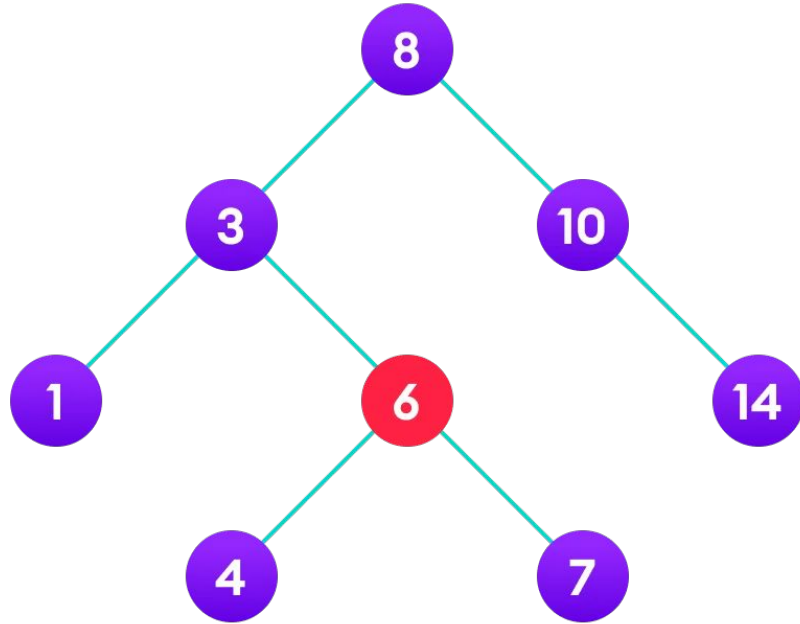
4 is not found so, traverse through the left subtree of 8



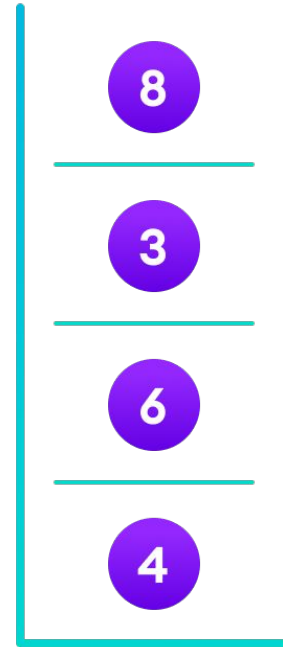
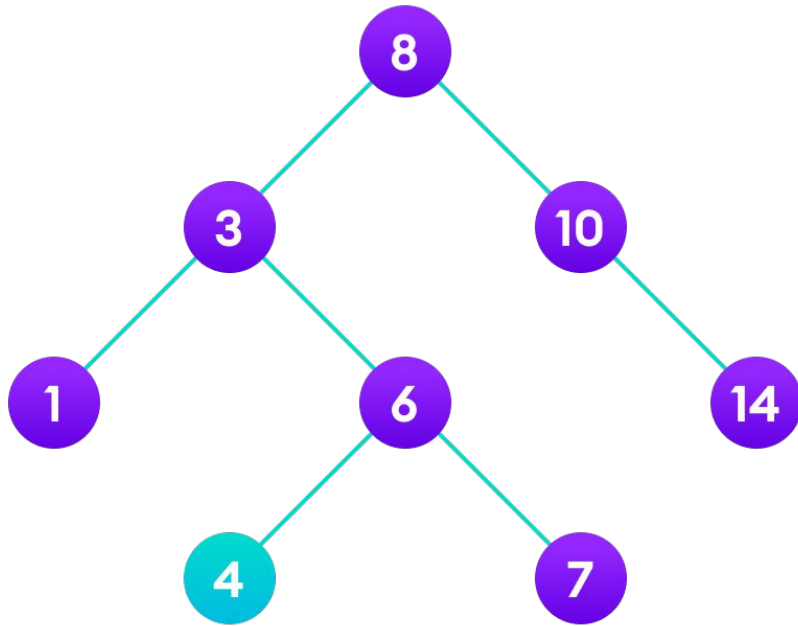
4 is not found so, traverse through the right subtree of 3



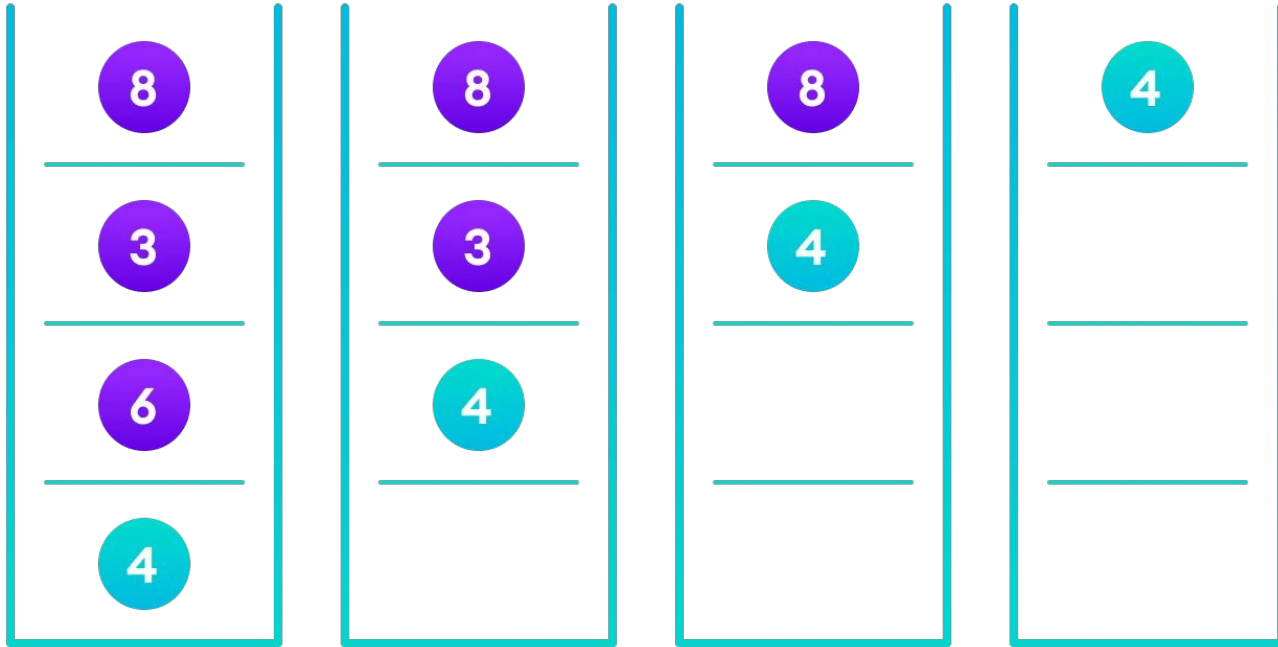
4 is not found so, traverse through the left subtree of 6



4 is found



If the value is found in any of the subtrees, it is propagated up so that in the end it is returned, otherwise null is returned



Algorithm

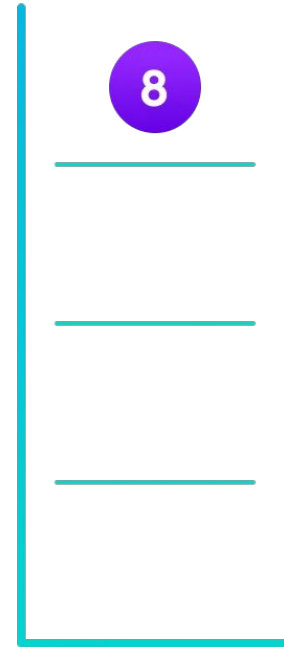
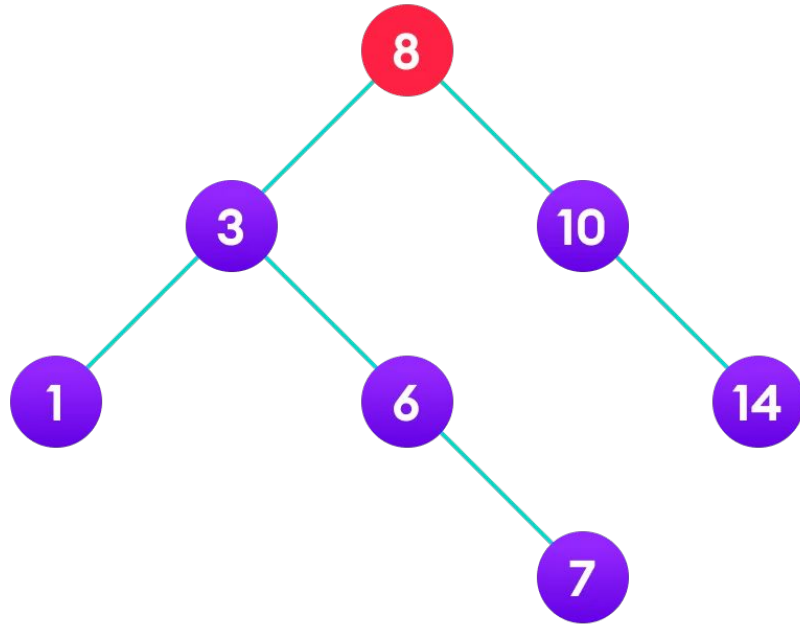
```
def search(root):  
    if root == None:  
        return None  
    if number == root.val:  
        return root.val  
    if number < root.val:  
        return search(root.left)  
    if number > root.val:  
        return search(root.right)
```

Inserting Operation

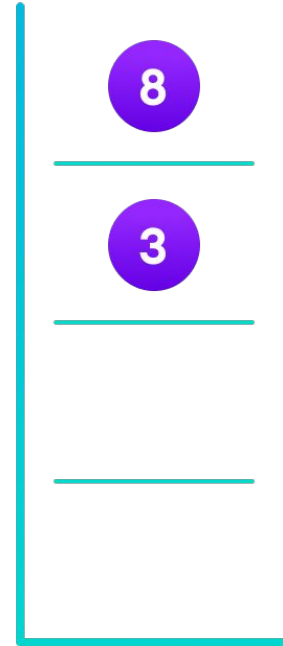
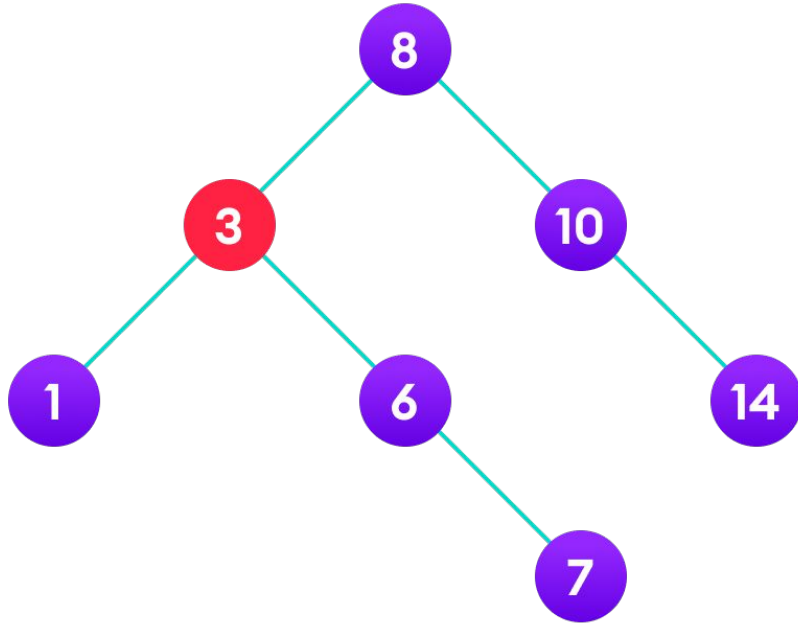
- Inserting a value in the correct position is similar to searching because we try to maintain the rule that the left subtree is lesser than root and the right subtree is larger than root.
- We keep going to either right subtree or left subtree depending on the value and when we reach a point left or right subtree is null, we put the new node there.

Let's try to visualize how we add a number 4 to an existing BST.

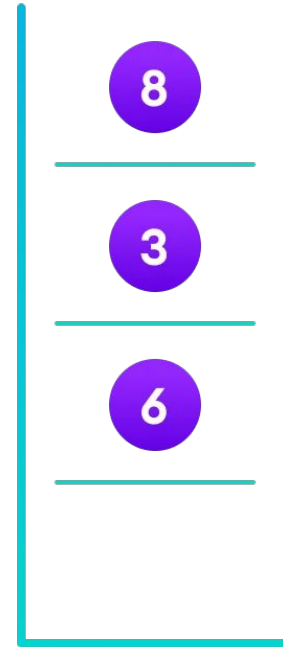
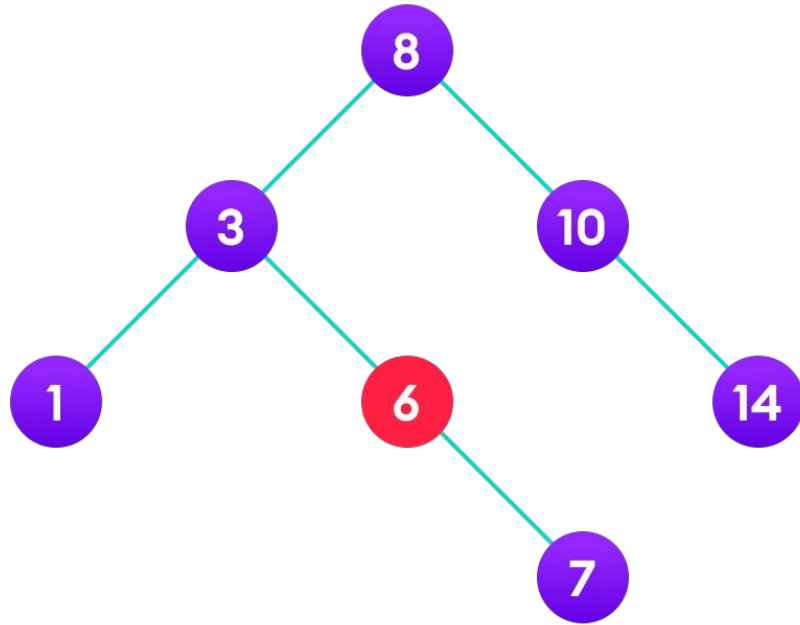
$4 < 8$ so, transverse through the left child of 8



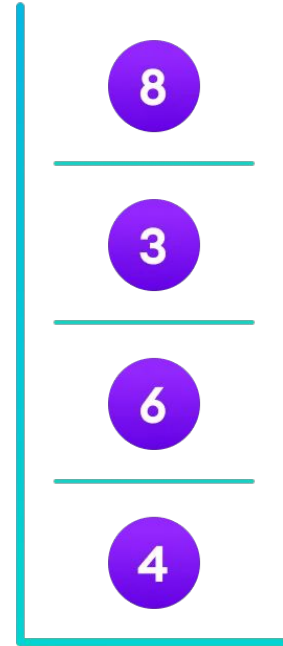
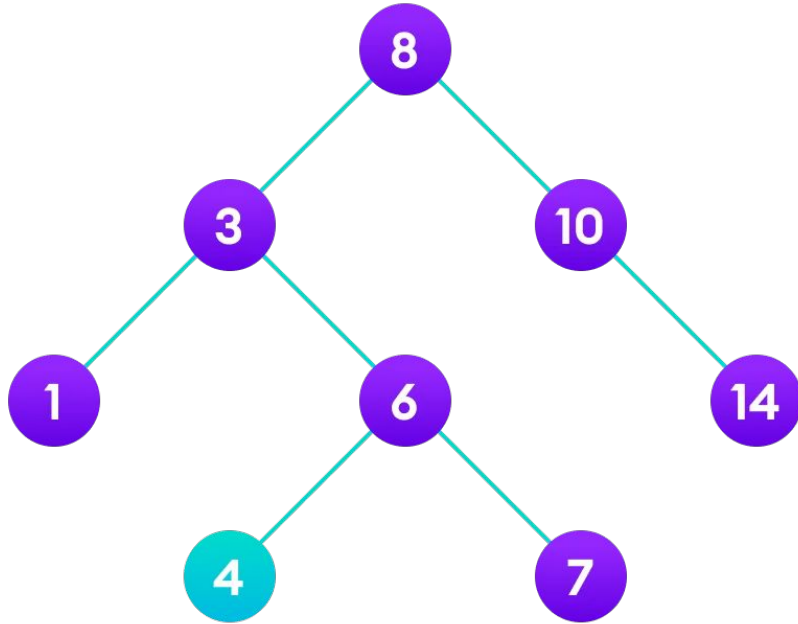
$4 > 3$ so, transverse through the right child of 8

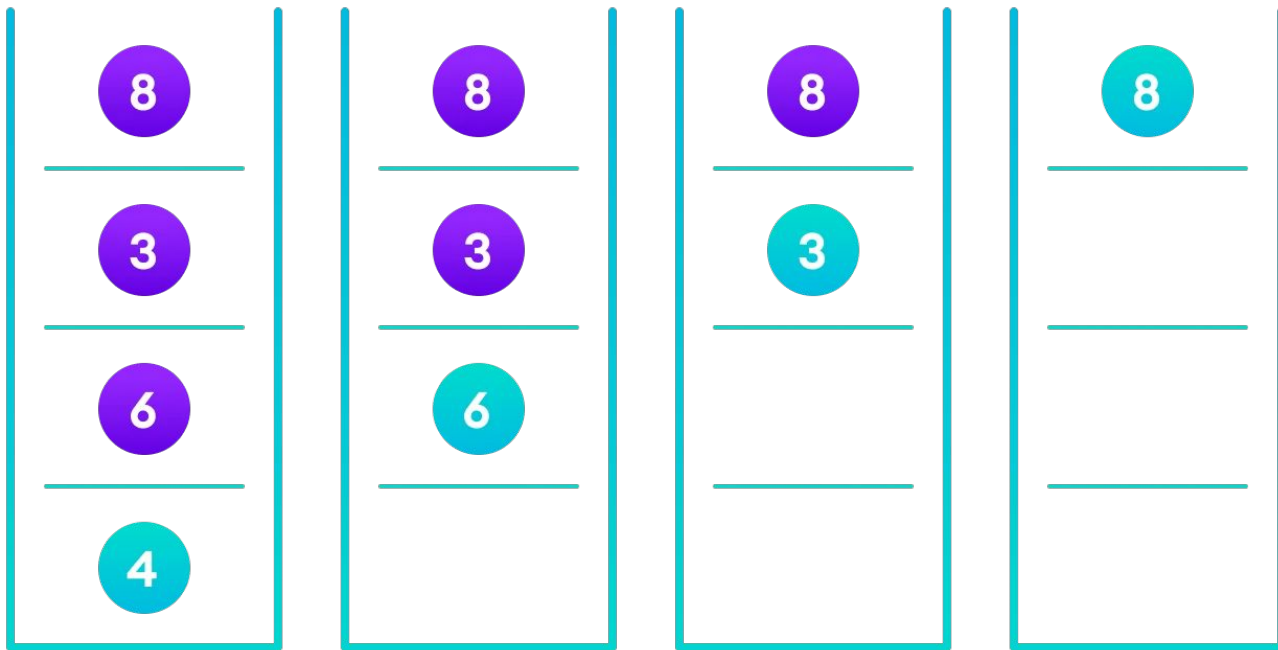


$4 < 6$ so, transverse through the left child of 6



Insert 4 as a left child of 6





Algorithm

```
def insert(node):
```

```
    if node == None
```

```
        return createNode(data) ## assume createNode creates a new node to be returned
```

```
    if (data < node.data)
```

```
        node.left = insert(node.left, data);
```

```
    elif (data > node->data)
```

```
        node.right = insert(node.right, data);
```

```
    return node
```

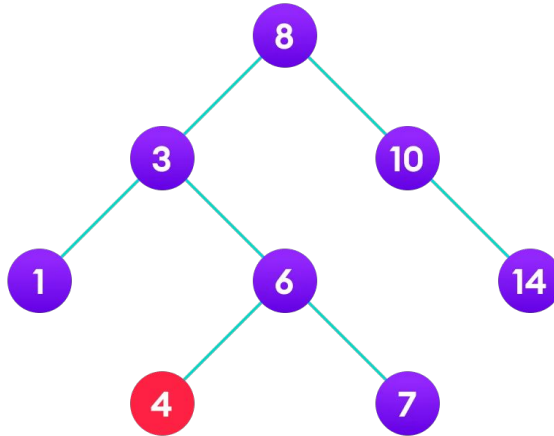
CheckPoint

Delete Node in a BST

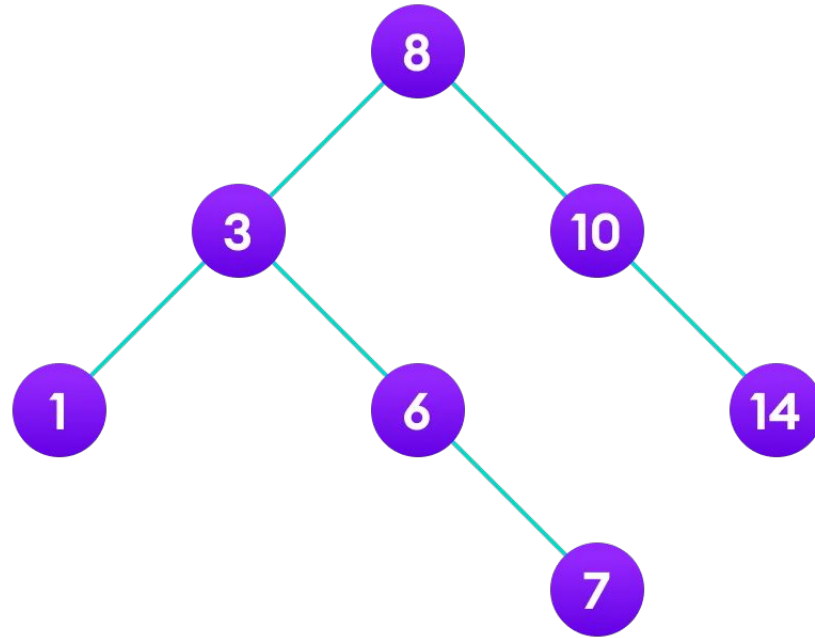
Deletion Operation

- There are three cases for deleting a node from a binary search tree.

Case One: In the first case, the node to be deleted is the leaf node. In such a case, simply delete the node from the tree. 4 is to be deleted.



Delete the node

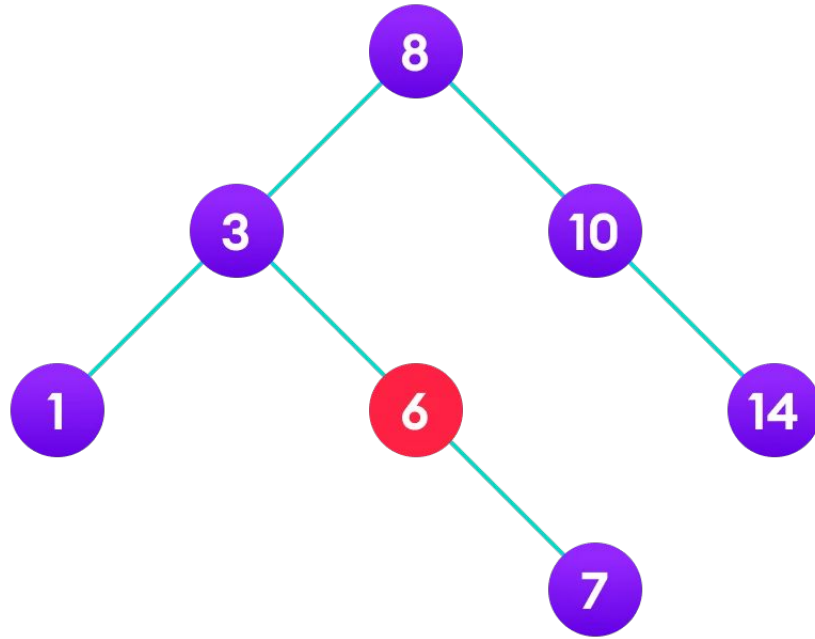


Case Two

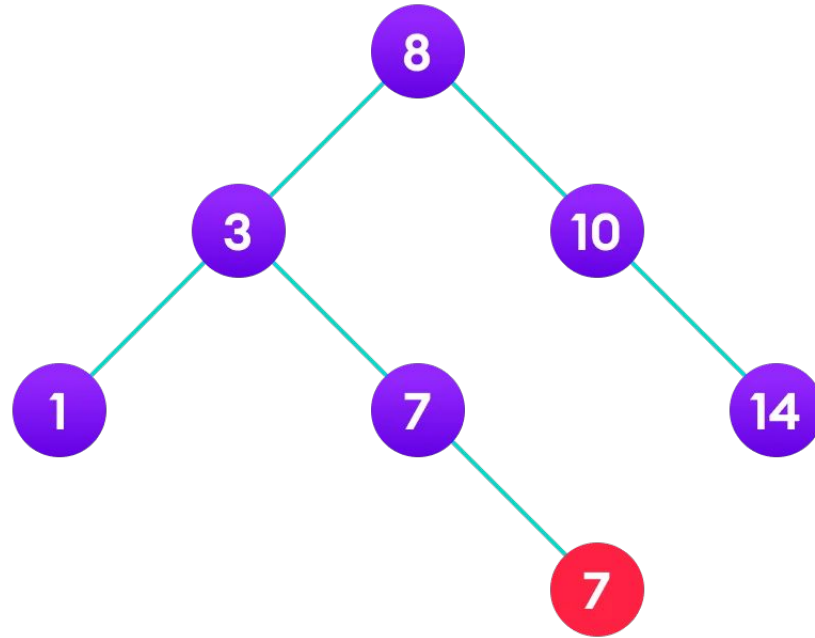
In the second case, the node to be deleted lies has a single child node. In such a case follow the steps below:

1. Replace that node with its child node.
2. Remove the child node from its original position.

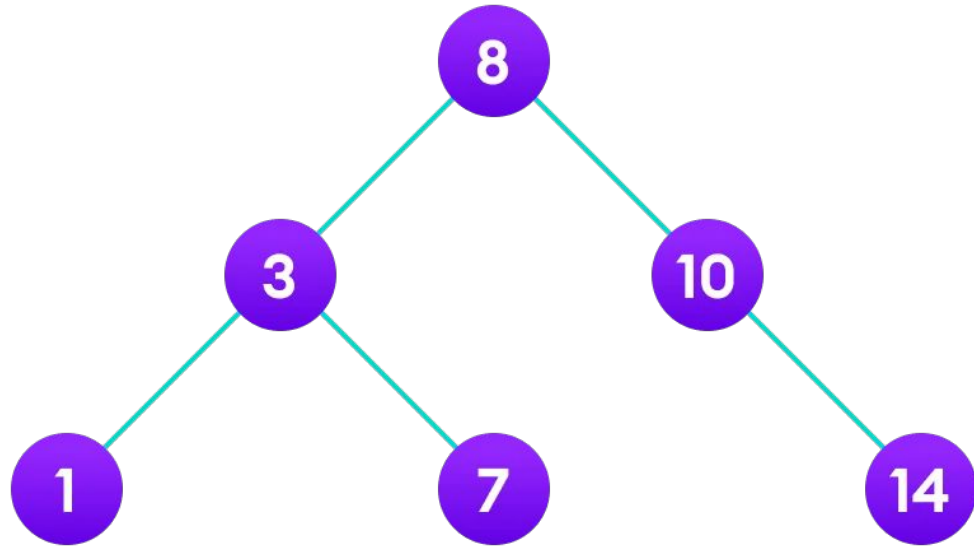
6 is to be deleted



copy the value of its child to the node and delete the child



Final tree

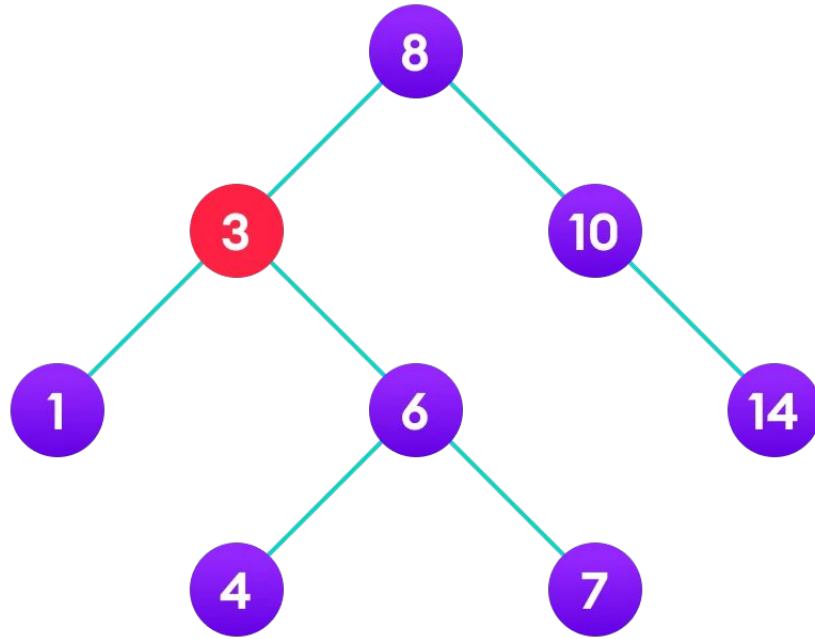


Case Three

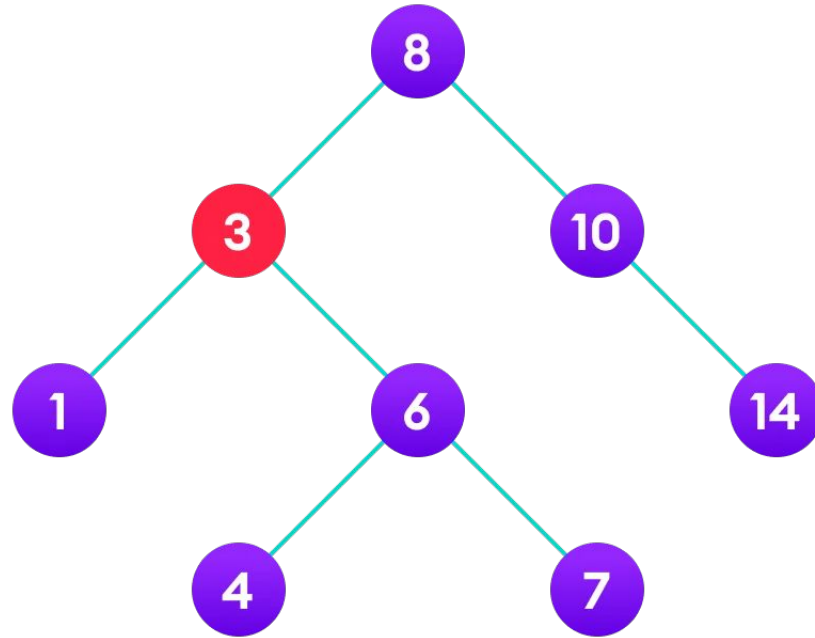
In the third case, the node to be deleted has two children. In such a case follow the steps below:

1. Get the inorder successor of that node.
2. Replace the node with the inorder successor.
3. Remove the inorder successor from its original position.

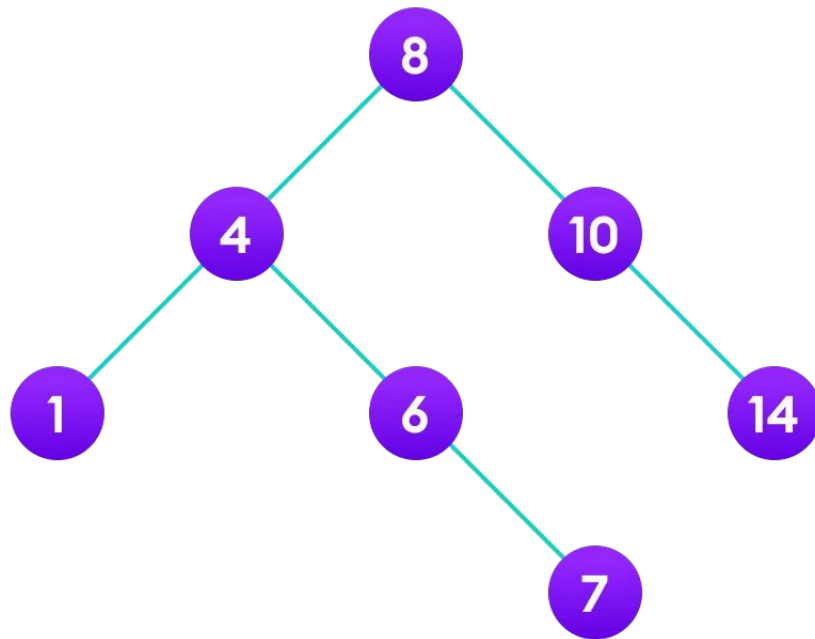
3 is to be deleted



Copy the value of the inorder successor (4) to the node



Delete the inorder successor



```

def deleteNode(root, key):
    # Return if the tree is empty
    if not root:
        return root
    # Find the node to be deleted
    if key < root.key:
        root.left = deleteNode(root.left, key)
    elif(key > root.key):
        root.right = deleteNode(root.right, key)
    else:
        # If the node is with only one child or no child
        if not root.left:
            return root.right
        elif not root.right:
            return root.left

        # If the node has two children,
        # place the inorder successor in position of the node to be
deleted
        temp = minValueNode(root.right)
        root.key = temp.key

        # Delete the inorder successor
        root.right = deleteNode(root.right, temp.key)

    return root

```

```

# Insert a node
def insert(node, key):

    # Return a new node if the tree is
empty
    if not node:
        return createNode(key)

    # Traverse to the right place and
insert the node
    if key < node.key:
        node.left = insert(node.left, key)
    else:
        node.right = insert(node.right,
key)

    return node

# Find the inorder successor
def minValueNode(node):
    current = node

    # Find the leftmost leaf
    while(current.left):
        current = current.left

    return current

```

Time and Space Complexity Analysis

Binary Tree

- Traversing
 - Time = ?
- Searching
 - Time = ?
- Insertion
 - Time = ?
- Deletion
 - Time = ?
- Space = ?

Binary Search Tree

- Traversing
 - Time = ?
- Searching
 - Time = ?
- Insertion
 - Time = ?
- Deletion
 - Time = ?
- Space = ?

Time and Space Complexity Analysis

Binary Tree

- Traversing
 - Time = $O(n)$
- Searching
 - Time = $O(n)$
- Insertion
 - Time = $O(n)$
- Deletion
 - Time = $O(n)$
- Space = $O(n)$... Why?

Binary Search Tree

- Traversing
 - Time = $O(h)$ where h is the height of BST.
- Searching
 - Time = $O(h)$
- Insertion
 - Time = $O(h)$
- Deletion
 - Time = $O(h)$
- Space = $O(n)$

Common Pitfalls

- Null pointer
 - ❑ `node.right <> check existence of node`
 - ❑ `node.left <> check existence of node`

Applications of a Tree

1. Representation structure in **File Explorer**. (Folders and Subfolders) uses N-ary Tree.
2. **Auto-suggestions** when you google something using Trie.
3. Used in **decision-based machine learning algorithms**.
4. Tree forms the backbone of other complex data structures like heap, priority queue, spanning tree, etc.
5. A binary tree is used in **database indexing** to store and retrieve data in an efficient manner.
6. To implement **Heap** data structure.
7. **Binary Search Trees (BST)** can be used in **sorting algorithms**.

Practice Questions

- [Lowest Common Ancestor of a Binary Tree](#)
 - [Validate Binary Search Tree](#)
 - [Kth Smallest Element in a BST](#)
- [Maximum Sum BST in Binary Tree](#)

Quote

“A journey of a thousand miles begins with a single step.”

-Lao Tzu